✅ PHASE 1: Setup & Fundamentals

---

1. Introduction to React

Explanation:
React is a **JavaScript library for building user interfaces**. It focuses on creating reusable UI components and efficiently updating the DOM using a **Virtual DOM**.

Why React?

- Component-based architecture
- Fast rendering with Virtual DOM
- Huge ecosystem and community support

How It Works:
React maintains a **Virtual DOM** (a lightweight copy of the real DOM). When state changes:

1. React updates the Virtual DOM.
2. It compares the Virtual DOM with the previous version (Diffing).
3. Only the changed parts are updated in the real DOM (Reconciliation).

Diagram (Virtual DOM Concept):

```
[Component State Change] → [Virtual DOM Update] → [Diffing Algorithm] → [Real DOM Update]
```

Code Example:

JSX
function App() {
return <h1>Hello, React!</h1>;
}
Show more lines

Best Practices:

- Use **functional components** and **hooks** (modern React).
- Keep components small and reusable.

Pitfalls:

- Direct DOM manipulation (avoid using `document.getElementById` inside React).
- Forgetting to return JSX from a component.

Mini Project:
Create a simple **Greeting App** that displays “Hello, React!” and changes text when a button is clicked.

Advanced Insight:
React 18 introduced **Concurrent Rendering** and **Automatic Batching** for better performance.

---

2. Setting Up Environment

- Install **Node.js** and **npm**.
- Create a React app:

Shell
npx create-react-app my-app
cd my-app
npm start
Show more lines

- Alternative: **Vite** for faster builds:

Shell
npm create vite@latest my-app

Show more lines

---

3. JSX & Rendering

Explanation:
JSX is a syntax extension that lets you write HTML-like code inside JavaScript.

How It Works:
JSX is compiled by **Babel** into `React.createElement()` calls.

Example:

JSX
function App() {
const name = "Pritam";
return <h1>Hello, {name}!</h1>;
}
Show more lines

Best Practices:

- Wrap multiple elements in a fragment (`<>...</>`).
- Use `className` instead of `class`.

Pitfalls:

- Forgetting `key` in lists.
- Mixing HTML attributes incorrectly.

Mini Project:
Create a **Dynamic Greeting App** that changes based on time of day.

Advanced Insight:
JSX is not HTML; it's syntactic sugar for `React.createElement()`.

---

4. Components & Props

Explanation:
Components are reusable building blocks. Props allow passing data from parent to child.

Example:

JSX
function Greeting({ name }) {
return <h1>Hello, {name}!</h1>;
}

function App() {
return <Greeting name="Pritam" />;
}

Show more lines

Best Practices:

- Keep components small and focused.
- Use **prop-types** or TypeScript for type safety.

Pitfalls:

- Overusing props (consider Context API for global state).

Mini Project:
Create a **Profile Card Component** that accepts `name`, `age`, and `bio` as props.

---

5. State & Event Handling

Explanation:
State represents dynamic data in a component. Events handle user interactions.

Example:

JSX
```
function Counter() {
const [count, setCount] = React.useState(0);
return (
<div>
<p>{count}</p>
<button onClick={() => setCount(count + 1)}>Increment</button>
</div>
);
}
```

Show more lines

Best Practices:

- Never mutate state directly (use `setState` or hooks).
- Use functional updates when needed:

JSX
```
setCount(prev => prev + 1);
```
Show more lines

Pitfalls:

- Forgetting to bind `this` in class components (not an issue in functional components).

Mini Project:
Create a **Counter App** with increment, decrement, and reset buttons.

PHASE 2: Hooks & Lifecycle

---

1. React Hooks Overview

Hooks allow you to use state and lifecycle features in **functional components**.

Why Hooks?

- Replace class components for most use cases.
- Make code cleaner and reusable.

---

2. `useState` Hook

Explanation:
Manages state in a functional component.

Example:

JSX
function Counter() {
const [count, setCount] = React.useState(0);
return (

{count}
<button onClick={() => setCount(count + 1)}>Increment</button>

);
}
Show more lines

Best Practices:

- Use functional updates for dependent state:

JSX
setCount(prev => prev + 1);
Show more lines

Pitfalls:

- State updates are asynchronous; don’t rely on immediate changes.

Mini Project:
Create a **Counter App** with increment, decrement, and reset buttons.

---

3. `useEffect` Hook

Explanation:
Handles **side effects** like data fetching, subscriptions, or DOM manipulation.

How It Works:
Runs after render. Can run:

- On every render
- Only on mount/unmount
- When dependencies change

Example:

JSX
useEffect(() => {
console.log("Component mounted or updated");
return () => console.log("Cleanup on unmount");
}, []);
Show more lines

Best Practices:

- Always specify dependencies.
- Use cleanup for subscriptions.

Pitfalls:

- Missing dependencies can cause bugs.
- Infinite loops if dependencies are wrong.

Mini Project:
Create a **Timer App** that starts on mount and stops on unmount.

---

4. Additional Hooks

- **`useRef`**: Access DOM elements or persist values without re-render.
- **`useMemo`**: Memoize expensive calculations.
- **`useCallback`**: Memoize functions to prevent unnecessary re-renders.

Example:

JSX
const memoizedValue = useMemo(() => expensiveCalculation(num), [num]);

Show more lines

---

5. Custom Hooks

Explanation:
Reusable logic extracted from components.

Example:

JSX
function useWindowWidth() {
const [width, setWidth] = useState(window.innerWidth);
useEffect(() => {
const handleResize = () => setWidth(window.innerWidth);
window.addEventListener('resize', handleResize);
return () => window.removeEventListener('resize', handleResize);
}, []);
return width;
}
Show more lines

Mini Project:
Create a **useFetch Hook** for API calls.

✅ PHASE 3: Routing & Context API

---

1. React Router

Explanation:
React Router is used for **client-side routing** in React applications. It allows navigation between different components without reloading the page.

Why It Matters:
Single Page Applications (SPAs) need routing to display different views without full page refresh.

Core Concepts:

- **BrowserRouter**: Wraps the app and enables routing.
- **Route**: Defines a path and the component to render.
- **Link**: Navigation without page reload.
- **useNavigate**: Programmatic navigation.
- **Dynamic Routes**: Routes with parameters.
- **Nested Routes**: Routes inside routes.

Diagram (React Router Flow):

```
[BrowserRouter]
    ├── [Route path="/"] → Home Component
    ├── [Route path="/about"] → About Component
    └── [Route path="/user/:id"] → User Component
```

Example:

JSX
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
return (
<BrowserRouter>
<nav>
<Link to="/">Home</Link>
<Link to="/about">About</Link>
</nav>
<Routes>
<Route path="/" element={<Home />} />
<Route path="/about" element={<About />} />
</Routes>
</BrowserRouter>
);
}

Best Practices:

- Use `Routes` instead of `Switch` (React Router v6).
- Use `useNavigate` for redirects instead of `history.push`.

Pitfalls:

- Forgetting to wrap routes inside `BrowserRouter`.
- Incorrect path matching.

Mini Project:
Build a **Multi-Page Blog App** with Home, About, and Blog Details pages using dynamic routes.

---

2. Context API

Explanation:
Context API provides a way to share data across components without **prop drilling**.

Why It Matters:
Avoid passing props through multiple levels when many components need the same data.

How It Works:

- Create a context using `React.createContext()`.
- Provide context value using `Context.Provider`.
- Consume context using `useContext()`.

Diagram (Context Flow):

```
[Context Provider]
    ├── Component A
    ├── Component B
        └── Component C (consumes context)
```

Example:

JSX
const ThemeContext = React.createContext();

function App() {
return (
<ThemeContext.Provider value="dark">
<Toolbar />
</ThemeContext.Provider>
);
}

```
function Toolbar() {
const theme = React.useContext(ThemeContext);
return <div>Current theme: {theme}</div>;
}
```

Best Practices:

- Use Context for **global state** like theme, authentication.
- Avoid overusing Context for everything (use Redux for complex state).

Pitfalls:

- Updating context frequently can cause unnecessary re-renders.

Mini Project:
Create a **Theme Switcher App** using Context API.

✅ PHASE 4: Redux Toolkit & Performance Optimization

---

1. Redux Toolkit Overview

Explanation:
Redux is a **state management library** for predictable state updates. Redux Toolkit (RTK) is the modern, recommended way to use Redux because it reduces boilerplate and simplifies logic.

Why Redux Toolkit?

- Simplifies Redux setup.
- Built-in support for **Immer** (immutable updates).
- Includes **createSlice**, **configureStore**, and **createAsyncThunk**.

Redux Flow Diagram

```
[Component Dispatches Action] → [Reducer Updates State] → [Store Holds State] → [Component Reads State]
```

---

2. Core Concepts

- **Store**: Centralized state container.
- **Slice**: A piece of state + reducers.
- **Actions**: Describe what happened.
- **Reducers**: Update state based on actions.
- **Selectors**: Read data from the store.

---

3. Setting Up Redux Toolkit

Example:

JSX
import { configureStore, createSlice } from '@reduxjs/toolkit';

// Slice
const counterSlice = createSlice({
name: 'counter',
initialState: { value: 0 },
reducers: {
increment: state => { state.value += 1; },

```
decrement: state => { state.value -= 1; }
}
});

export const { increment, decrement } = counterSlice.actions;

// Store
const store = configureStore({
reducer: { counter: counterSlice.reducer }
});

default store;
```
Show more lines

Usage in Component:

```
JSX
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement } from './store';

function Counter() {
const count = useSelector(state => state.counter.value);
const dispatch = useDispatch();

return (
<div>
<p>{count}</p>
<button onClick={() => dispatch(increment())}>+</button>
<button onClick={() => dispatch(decrement())}>-</button>
</div>
);
}
```
Show more lines

---

4. AsyncExplanation:

`createAsyncThunk` handles API calls and async operations.

Example:

```
JSX
import { createAsyncThunk } from '@reduxjs/toolkit';

export const fetchUsers = createAsyncThunk('users/fetch', async () => {
const response = await fetch('https://jsonplaceholder.typicode.com/users');
return response.json();
});
```
Show more lines

Best Practices

- Use **Redux Toolkit** instead of plain Redux.
- Keep slices small and focused.
- Use **RTK Query** for data fetching.

Pitfalls

- Overusing Redux for local state (use hooks for component-level state).
- Forgetting to wrap app with `<Provider>`.

Mini Project:
Build a **Todo App** with Redux Toolkit and add async API calls for fetching todos.

✅ Performance Optimization in React

Why Optimize?

- Prevent unnecessary re-renders.
- Improve app speed for large-scale apps.

Techniques

1. **Memoization**
   - `React.memo` for components.
   - `useMemo` for expensive calculations.
   - `useCallback` for memoizing functions.

Example:

JSX
const memoizedValue = useMemo(() => computeExpensiveValue(num), [num]);

Show more lines

2. **Code Splitting**
   - Use `React.lazy` and `Suspense` for dynamic imports.

JSX
const LazyComponent = React.lazy(() => import('./LazyComponent'));

Show more lines

3. **Avoid Inline Functions**

    - Use `useCallback` for stable references.
4. **Virtualization**
    - Use libraries like `react-window` for large lists.

Mini Project:
Create a **Large List Rendering App** and optimize it using virtualization and memoization.

✅ PHASE 5: Testing & Deployment

---

1. Testing in React

Why Testing Matters:
Testing ensures your components work as expected and prevents regressions when code changes.

Types of Testing:

- **Unit Testing**: Test individual components.
- **Integration Testing**: Test how components work together.
- **End-to-End (E2E) Testing**: Test the entire app flow.

Tools for Testing

- **Jest**: JavaScript testing framework.
- **React Testing Library (RTL)**: Focuses on testing UI behavior, not implementation details.
- **Cypress**: For E2E testing.

React Testing Library Basics

Installation:

Shell
npm install --save-dev @testing-library/react @testing-library/jest-dom
Show more lines

Example Test:

JSX
import { render, screen } from '@testing-library/react';
import App from './App';

test('renders hello message', () => {
render(<App />);
const element = screen.getByText(/Hello, React!/i);
expect(element).toBeInTheDocument();
});
Show more lines

Best Practices:

- Test user interactions, not internal state.
- Use `screen` queries like `getByText`, `getByRole`.

Pitfalls:

- Avoid testing implementation details (e.g., internal hooks).
- Don’t over-test trivial things.

Mini Project:
Write tests for a **Counter Component**:

- Check initial count.
- Simulate button clicks and verify count updates.

E2E Testing with Cypress

Installation:

Shell
npm install --save-dev cypress

Show more lines

Run Cypress:

Shell
npx cypress open
Show more lines

Example:

JavaScript
describe('App Navigation', () => {
it('should navigate to About page', () => {
cy.visit('/');
cy.contains('About').click();
cy.url().should('include', '/about');
});
});
Show more lines

---

2. Deployment

Why Deployment Matters:
Your app needs to be accessible to users on the web.

Build Process

For Create React App:

Shell
npm run build
Show more lines

This creates an optimized production build in the `build` folder.

Deployment Options

- **Netlify**:
  - Drag and drop the build folder or connect GitHub repo.
- **Vercel**:
  - Install Vercel CLI or connect GitHub repo.
- **GitHub Pages**:
  - Use `gh-pages` package for deployment.

Best Practices:

- Use environment variables for API keys.
- Optimize images and bundle size before deployment.

Mini Project:
Deploy your **Blog App** or **Todo App** on Netlify or Vercel.

✅ OPTIONAL PHASE: TypeScript & Advanced Patterns

---

1. React with TypeScript

Why TypeScript?
TypeScript adds **static typing** to JavaScript, making React apps more robust and easier to maintain.

Benefits:

- Catch errors at compile time.
- Better IDE support and autocomplete.
- Easier collaboration in large projects.

Setup

Install TypeScript in a React project:

Shell
npm install typescript @types/react @types/react-dom

Show more lines

Or use:

Shell
npx create-react-app my-app --template typescript
Show more lines

Typing Props

Example:

TypeScript
type GreetingProps = {
name: string;
age?: number; // optional
};

function Greeting({ name, age }: GreetingProps) {
return <h1>Hello, {name}! {age && `Age: ${age}`}</h1>;
}
Show more lines

Typing State

TypeScript
const [count, setCount] = useState<number>(0);
Show more lines

Typing Hooks

TypeScript
const inputRef = useRef<HTMLInputElement>(null);

Show more lines

Best Practices:

- Use `interface` or `type` for props.
- Avoid `any` type; use `unknown` or proper types.

Mini Project:
Convert your **Todo App** to TypeScript.

---

2. Advanced React Patterns

These patterns help build **reusable and scalable components**.

---

a) Higher-Order Components (HOC)

Explanation:
A function that takes a component and returns a new component with added functionality.

Example:

JSX
function withLogger(WrappedComponent) {
return function(props) {
console.log('Props:', props);
return <WrappedComponent {...props} />;
};
}
Show more lines

---

b) Render Props

Explanation:
A technique where a component's child is a function that returns JSX.

Example:

JSX
function DataProvider({ render }) {
const data = { name: 'React' };
return render(data);
}

<DataProvider render={(data) => <h1>{data.name}</h1>} />

Show more lines

---

c) Compound Components

Explanation:
Components that work together to share state implicitly.

Example:

JSX
function Tabs({ children }) {
return {children} ;
}

Tabs.Tab = function({ label }) {
return <button>{label}</button>;
};

Show more lines

Best Practices:

- Use these patterns for **component libraries**.
- Avoid overcomplicating simple components.

Mini Project:
Build a **Custom Tabs Component** using Compound Components.